Email: John.dixon@hull.ac.uk
Meetings: https://tinyurl.com/jdixonhull
Feedback: https://tinyurl.com/jdhullfeedback
Office: RBB-315

Real-Time Services

Distributed Systems Programming

Dr John Dixon

UNIVERSITY of HULL

Overview

We are looking at these topics today:

- What are real-time services and what are they for?
- How do they differ from other services?
- What additional considerations do we need to make when using real-time services?

UNIVERSITY of HULL

What are Real-Time Services?

A real-time service provides data from author to recipient(s) immediately as the data is produced rather than waiting for the recipient to ask for it.

These types of service aim to:

- minimise the delay between sending and receiving a message
- whilst still achieving high quality and usability for the user

Generally this is achieved by keeping a connection open, minimising the number of operations that need to be completed between sending and receiving, and compressing data where possible.

UNIVERSITY of HULL

What are Real-Time Services?

Modern common examples are often specified as:

- Video and/or voice chat (e.g. Discord)
- Live video streaming (e.g. Twitch)

But real-time services are used in many different ways, for example:

- Text-chat
- Live-updating content (news/betting)
- Always-on sensors
- Collaborative applications (e.g. Google Docs)

UNIVERSITY of HULL

Request/Response vs. Real-Time

We have seen the Request/Response mechanism used in REST and the vast majority of web services.

This design typically closes the connection after the response is received

- That's useful because a server can only handle a certain number of concurrent connections.
- When we are expecting clients to request what they need we can close the connection once we have fulfilled their request.
- This allows our server to move onto another request with the released system resources.

UNIVERSITY of HULL

Request/Response vs. Real-Time

Request/Response is wasteful if the same client keeps making requests…

- Making a new connection has a resource overhead for both the client and server
- This is exacerbated if handshakes or connection specifics need to be determined (e.g. for encryption) before the message(s) can actually be sent
- A stateless server will 'forget' about the client so may need to do a database lookup per request

It would be better if the connection was left open!

- Some servers will allow this – the connection is on a timeout so requests sent quickly one after another do not require recreating it. Once the timeout elapses the connection is closed.
- You can use an HTTP header (keep-alive) to ask the server to keep your connection open. The suggestion is that you are likely to send another request soon (many servers ignore this).
- Because the client cannot guarantee that the connection is going to be left open we often have to ASSUME that it has been closed and this means more work.

UNIVERSITY of HULL

Request/Response vs. Real-Time

Wait… would it *actually* be better if the connection was left open?

- Yes, if you have…
  - few clients who want to connect,
  - who regularly need new data,
  - and who need fast service
- No, if you have…
  - lots of clients who want to connect,
  - who only need data once or every now and again,
  - and where the speed of the service isn't essential

UNIVERSITY of HULL

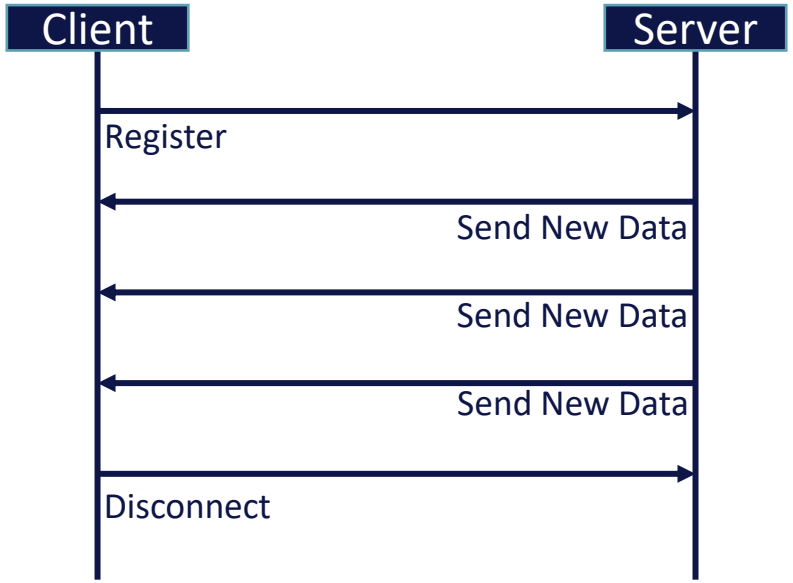
Request/Response vs. Real-Time

Even if the connection is left open, a request/response mechanism

still requires the client to request data before a response is given

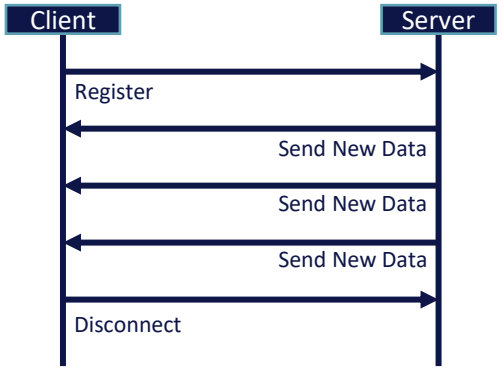
If we want ALL of the data that the server has to send us then it is much more efficient to register our interest (similar to an event handler in C#) and then wait for data as an when it is ready.

We can then disconnect when we have had enough.



UNIVERSITY of HULL

Registering for New Data



This works really, really well when the data being sent to lots of users is exactly the same

- One to many connections (e.g. Twitch/Betting)
  - The same live video is sent to many viewers
  - The same odds are updated to many betting terminals at the same time
- Many to many connections (e.g. Discord/Teams)
  - Data received from each client is sent back out to all other clients

It is less efficient when you have many different pieces of data being sent to or received from different clients (e.g. IoT)

- Sensors each sending their own live-sensed data
- This requires more work for the server to manage each connection and the data streams independently

UNIVERSITY of HULL

How?

At the socket level, real-time services tend to use UDP sockets ***where possible*** because these don't require the same level of per-message overhead as other options (e.g. TCP)

However, UDP has some side-effects…

- No error recovery
- No acknowledgement of data received successfully
- No assurances that messages are received in the right order
- UDP is 'connectionless' – there is no warning that data will be sent or that it has finished

Many real-time systems overcome these issues (where they cause a problem) by using a protocol (or protocols) which *use* UDP but add additional features at a higher level.

Connection Managed by the Protocol

As UDP doesn't manage a connection, a real-time protocol should do this for us.

This usually means:

- The two sides send control messages in addition to the real-time data stream
  - Often via a more reliable mechanism such as REST (using TCP)
  - An example might be a set-up message sent by a client asking for a real-time connection to be established, identifying what data it wants/will send, how and where
- The connection is maintained by the real-time protocol, e.g.
  - Participants are usually expected to identify when they no longer want to be connected
  - If data stops being received unexpectedly, control messages may be sent to determine the cause of the problem
  - A client may need to 'check-in' regularly with a control message to let the server know it's still listening (otherwise the server should unregister the client)
  - Data loss may be determined where required (we'll come onto this)

UNIVERSITY of HULL

Important Considerations for Real-Time Services

UNIVERSITY of HULL

Synchronisation

Often multiple data sources are sent simultaneously

E.g. Audio and video from many different people in an online meeting

Audio/video can get out of sync easily:

- 0 packets of audio may be sent in a silent part of a video – need to identify how long the silence will be
- a number of packets of audio/video may not send or be received correctly. Someone with a poor internet connection may cause lots of packet loss but their video stream needs to stay synchronised with everyone else
- packets/messages may not arrive in the order that they were sent

Timestamps and Sequencing

Some of the issues with synchronisation can often be solved to some degree with the use of timestamps, sequence numbers and references.

Timestamps:

- The exact time should be agreed by all participants
- Timestamp is sent with the data to identify when it was sent
- The audio/video/messages can be synchronised with one another based on the timestamps

Sequence Numbers:

- A number is applied to packet of data sent
- This identifies the order of the packets so if one is delayed it can be inserted in the correct place
- Any missing pieces of data can be identified if there is a gap in the sequence numbers

References:

- Data from one stream can reference the timestamp or sequence number from a different stream to ensure they are played concurrently

UNIVERSITY of HULL

Frames

One type of sequence number for audio and video is called ‘frames’

These are equal to the FPS (frames per second) of the video

For each feed audio may be split into equal length frames and synchronised with the video frames based on the frame number:

Data Stream:

Video Frame 1	Video Frame 2	Audio Frame 1	Video Frame 3	Audio Frame 2	Audio Frame 3

Output:

Video Frame 1	Video Frame 2	Video Frame 3
Audio Frame 1	Audio Frame 2	Audio Frame 3

UNIVERSITY of HULL

Buffers

Even with a very fast connection data may be delayed.

Therefore it is often important to keep a buffer of data and play back the buffer to the user slightly (or much) later than when it was received.

A buffer:

- Helps in situations where there is a temporary slow-down in connection speed (there is still data in the buffer so this can continue to play out and the slowdown doesn't cause the feed to pause)
- Can ensure that packets received out of order have enough time to be received and put back into the correct order before being played to the user

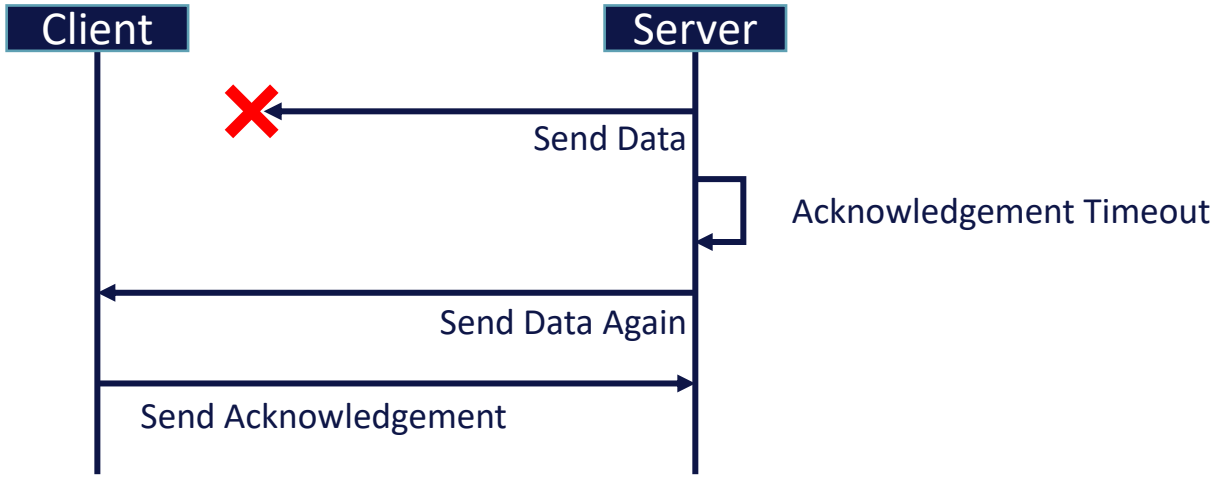
In non-live video applications (e.g. Netflix) a buffer:

- Allows maximisation of the connection speed to download the entire file in the background and play back at a normal speed, rather than trying to download at the rate the file should be played.

UNIVERSITY of HULL

Acknowledgement

Protocols with guaranteed data arrival use acknowledgement so that the sender knows the receiver has received each packet.

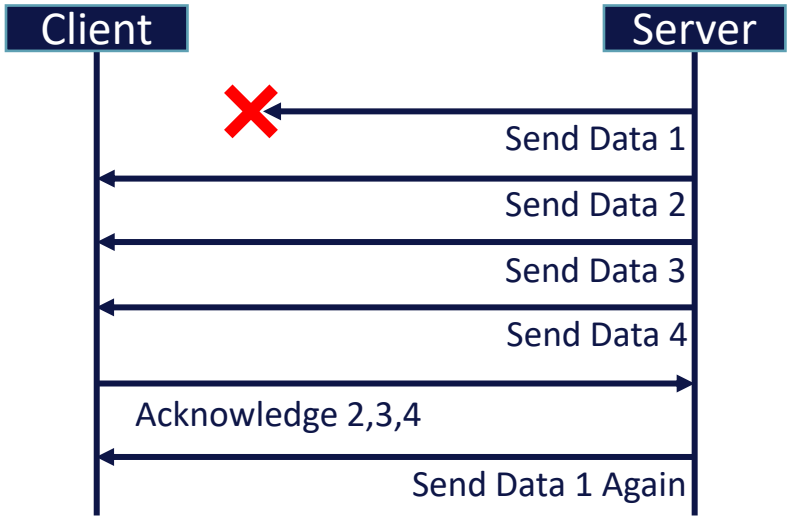


This process can slow down real-time systems because each packet needs acknowledgement and there may be MANY packets for video streaming.

UDP has no default acknowledgement mechanism.

Infrequent Acknowledgement

Real-time protocols that require no data loss may employ some form of infrequent acknowledgement where a larger number of packets are received before loss is identified. This is usually applied at the level of the real-time protocol rather than the transport protocol used.



UNIVERSITY of HULL

Voice and Video

Usually voice and video communication can cope with a small amount of packet loss.

- Small breaks in video and audio can be overlooked by users when they can still understand the person they are talking to
- Therefore acknowledgement can often be ignored altogether

Real problems start to occur when the data stream is being created by the producer faster than the consumer can download it

- Often the cause of this is too high quality from the producer
  - E.g. Producer sends video and audio at 120fps in 4K, consumer's Internet connection can only download video and audio at 20fps at 720p
- This can cause the video/audio to be played back with a large delay or fail completely

UNIVERSITY of HULL

Compression

Compression is important, especially for voice and video

Other data (such as sensor data) can be compressed too

Compression minimises the amount of bandwidth required to send the data by minimising the amount of data that is actually sent with the intention of ensuring little to no impact on the consumer.
There are many types of compression for different types of data.

Some examples:

- Reducing the number of colours that can be encoded. : all slightly different colours but you probably wouldn't notice a difference if they were all shown as the same colour
- Reducing the framerate/frame size
- Combining data into an average (e.g. sensor data, a number of nearby pixels or a very short portion of audio)

UNIVERSITY of HULL

Bandwidth

Less bandwidth usually requires more compression

Clever real-time protocols have mechanisms to modify the level of compression depending on how fast the client can consume the data

- If the client is building a large buffer quickly, send the data in higher quality (less compression, more data to send)
- If the client is consuming its buffer faster than it is creating it, send the data in lower quality (more compression, less data to send)

The server may also choose to lower the quality if its own bandwidth is limited (e.g. it has lots of current connections)

Peer-To-Peer

Where the connection is only between two machines (e.g. not a meeting with more than two people) it is sometimes better to establish a peer-to-peer connection directly between the two devices rather than using a server in-between

This can significantly speed up the connection as it has fewer steps from end-to-end.

- WebRTC uses a STUN Server to identify the public IP address of the device, allowing you to advertise this directly to the other device via an intermediary (TURN server).
- If a peer-to-peer connection can be established this is fast and lightweight, and requires no more resources from the server
- If a peer-to-peer connection cannot be established (e.g. incompatibility) the TURN server acts as a middleman, managing the incoming and outgoing data streams from both sides

UNIVERSITY of HULL

Async Programming

Asynchronous programming techniques are key because applications don't know when data will be received

- Async allows the application to do other things whilst awaiting data
- When new data is received a handler is called to manage it

This works if real-time data is received in chunks (not a continuous stream).

- E.g. Live text chat, batched data.
- Consider the use of YIELD

Otherwise, we might need a background thread to receive a continuous stream of data to the buffer. Then, the foreground thread reads from the buffer.

UNIVERSITY of HULL

Not Quite so Simple

Not all clients are born equal, some will block all UDP traffic, some will only accept/send encrypted traffic (TLS)

Firewalls and rules in devices along the way (e.g. routers) pose another risk to streams of data (especially data not sent via HTTPS)

Often real-time protocols need to have multiple ways to connect.

- If the fastest, most reliable connection method fails, try the next best… and so on…
- This can mean that servers managing the live data streaming need to work with many different clients, using many different technologies

UNIVERSITY of HULL

Efficiency and Scale

Large overheads maintaining lots of connections – especially with lots of data (e.g. video meetings)

Usually managed by dynamically allocating a meeting to a dedicated machine, VM or container

Usually still capped at a relatively small size for bidirectional connections (e.g. meetings capped to a few hundred max participants) compared to the huge scalability of non-real-time services.

UNIVERSITY of HULL

Key Words for Further Reading

- Real-time services
- Real-time networks
- Real-time protocols, including:
  - WebRTC,
  - RTP,
  - RTMP, and
  - RTSP
- Video and audio compression
- Asynchronous Programming

UNIVERSITY of HULL

Reflection

In this session we've looked at:

- What are real-time services and what are they for?
- How do they differ from other services?
- What additional considerations do we need to make when using real-time services?

In the next session we will look at Remote Procedure Calling

UNIVERSITY of HULL